MoE Pareto Frontier Oracle:

Understand TCO for Different Hardware Configurations

Kai Maeda

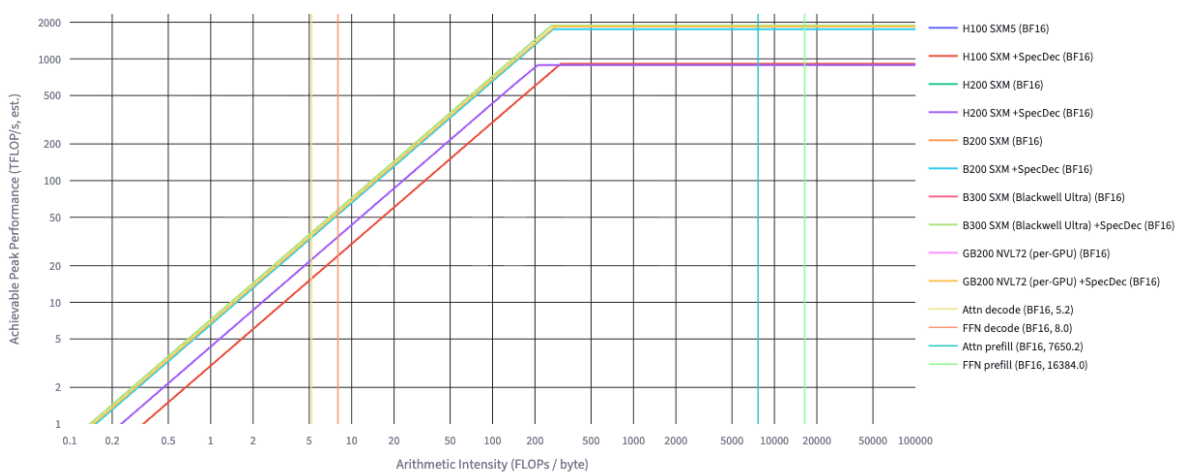
Introduction

Much of the motivations for this project are described in [4] so for a more comprehensive overview and background please refer to the InferenceX v2 paper.

With many of the frontier models transitioning to a disaggregated prefill/decode mixture of experts (MoE) models, there is great opportunity to optimize one's GPU clusters during inference. However depending on whether speculative decoding is used, the input sequence length (ISL), output sequence length(OSL), cluster utilization, tensor parallelism (TP), expert parallelism (EP), pipeline parallelism (PP), batch size, time to first token (TTFT), and inter token latency (ITL) constraints, the optimal GPU cluster can greatly change. The purpose of the MoE Pareto Frontier Oracle is to help users easily find the optimal GPU setup for their goals, budget, and performance constraints.

The TCO was initially attempted to be built from scratch but as the project continued, many holes were found. Iteratively it became more and more obviously that the current parameters that exist in InferenceX are very comprehensive. Thus my project slowly pivoted towards the baseline TCO calculator but with some extensions as described in the conclusion.

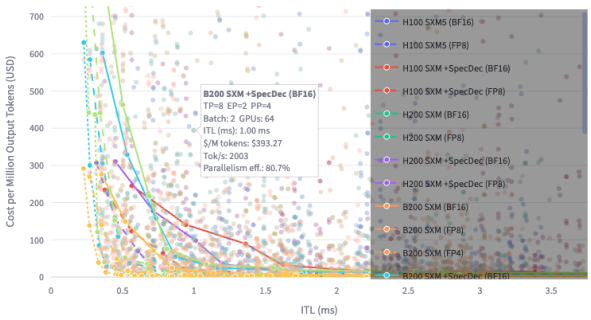
Roofline Chart



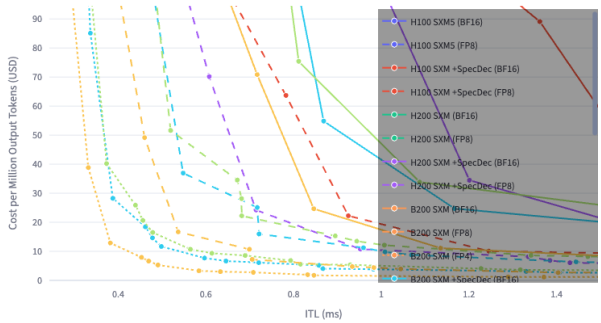
The purpose of the roofline chart is to better understand how the theoretical best performance for the various GPUs are and where the arithmetic intensity shifts from being memory bound to compute bound. As can be seen the operation (FFN and attention) does not matter as much as whether the operation is part of prefill or decode to determine compute vs memory bound.

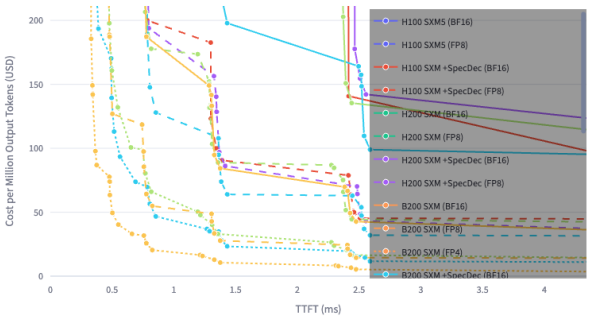
Source: Southpark

Pareto Frontier



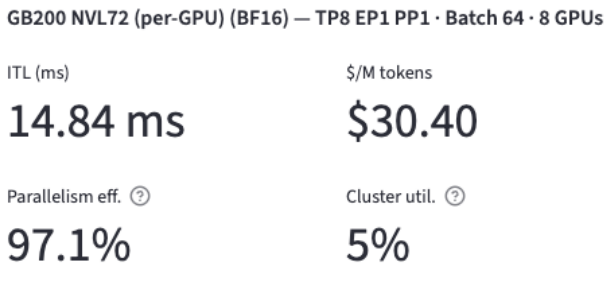
In order to calculate the Pareto Frontier curve, we sweeped across TP (1-8), PP (1-8), EP (1-8), batch sizes (1-128), and dropped suboptimal points from our Pareto Frontier curve.

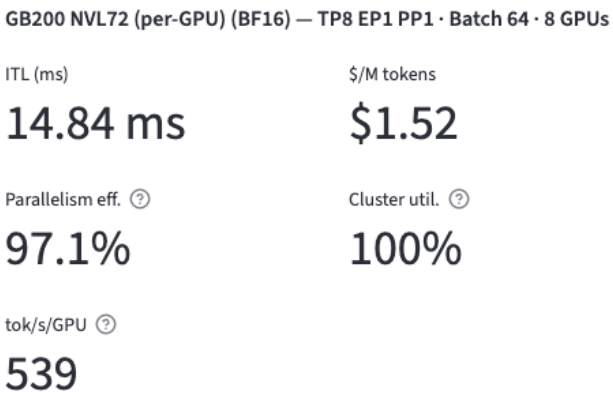




The Pareto Frontier curves can change depending on if we are prioritizing for ITL versus TTFT. Thus in a disaggregated prefill/decode setup it may be optimal to choose different GPUs for the prefill and decode phases.

Clicking on a point gives a comprehensive breakdown of the performance cost for that GPU. As we prioritize ITL/TTFT, we notice that the parallelism efficiency greatly decreases because we combine many parallelism techniques and pay high overhead costs. And as we prioritize low cost per million tokens, we achieve very high parallelism efficiency because we utilize less parallelism to reduce the communication overhead cost.





We also notice that as our clusters are being utilized for a larger percentage of the time, the amortized cost per million tokens decreases dramatically. The idea of how often the GPUs are being utilized is vital to deciding which hardware is optimal because a high initial cost GPU can be amortized by utilizing it heavily.

Limitations and Future Work

Known Limitations

1. Did not have a team of NVIDIA and AMD engineers to confirm the GPUs are running with an optimal setup so results only take into consideration optimizations mentioned in the introduction.
2. No easy access to the GPUs measured in the oracle, meaning that the results are theoretical rather than empirical
3. Claude Code was used to program the app so many assumptions were made to plot the Pareto frontier curve as described below.
4. Software efficiency for compute and bandwidth were assumed using fixed costs. This is to take into consideration the software tax that AMD pays for lagging behind in optimizing their software.

5. Batch size was fixed. In real systems the batch size is likely to be dynamic.
6. CapEx costs were assumed based off of the cost provider.
7. Speculative decoding speedups were assumed and hardcoded based off of the hardware.

Future Work

- Take historical software improvement per hardware to infer future TCO (Difficult to do accurately)
- Consider more advanced optimization techniques to better represent true capabilities of underlying hardware.
- Thoroughly check for accuracy of results against proven benchmarks.

Conclusion

Key Contributions:

1. Rebuilt TCO Calculator from InferenceX using Cost per Million Tokens for disaggregated prefill/decode
2. Used PP instead of DP to sweep through to find Pareto Frontier curve.
3. Added cluster utilization to better understand amortized cost based on GPU usage.
4. Roofline chart to better visualize preexisting GPU specs page on InferenceX. Incorporated operations (attention and FFN) into the chart.
5. X-axis for Pareto Frontier plot used inter token latency (ITL) and time to first token (TTFS) instead of target interactive tok/s/user. These were the metrics used in [5] and staying faithful to this papeer was prioritized.
6. Included parallelism efficiency to better understand how much communication overhead is causing a lack of parallelism

References

[1] Kimbo Chen, Dylan Patel, Daniel Nishball, et al. InferenceMAX: Open source inference benchmarking. SemiAnalysis, October 2025. Covers NVIDIA GB200 NVL72, AMD MI355X; metrics include throughput token per GPU, latency tok/s/user, cost per million tokens, tokens per provisioned megawatt; models: DeepSeek R1 670B, GPTOSS 120B, Llama3 70B.

[2] Trevor Gale, Deepak Narayanan, Cliff Young, and Matei Zaharia. MegaBlocks: Efficient sparse training with mixture-of-experts. *Proceedings of Machine Learning and Systems (MLSys)*, 5, 2023.

[3] Deepak Narayanan, Mohammad Shoeybi, Jared Casper, Patrick LeGresley, Mostofa Patwary, Vijay Anand Korthikanti, Dmitri Vainbrand, Prethvi Kashinkunti, Julie Bernauer, Bryan Catanzaro, Amar Phanishayee, and Matei Zaharia. Efficient large-scale language model training on GPU clusters using Megatron-LM. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC'21)*, 2021.

[4] Dylan Patel, Cam Quilici, Bryan Shan, et al. InferenceX v2: NVIDIA blackwell vs AMD vs hopper. SemiAnalysis, February 2026. Formerly InferenceMAX. Covers GB300 NVL72, MI355X, B200, H100; topics: disaggregated serving, wide expert parallelism, large MoE, SGLang, vLLM, TRT-LLM.

[5] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI'24)*, 2024.

Example FLOP Throughput Estimate With TP

Consider NVIDIA H100 SXM5 using BF16 FLOPs. Specs: 989 TFLOPS/s, 3.35 TB/s HBM, 900 GB/s NVLink per GPU. Consider the decode step with TP=4 on DeepSeek-V2 (236 Billion Params and 21 Billion Active Params, Hidden Dimension Size = 5120, 60 Transformer Layers). Hardcoded software efficiency to be 92% for compute and 90% for bandwidth. Let Batch size = 1, PP = 1, EP = 1, output sequence length = 1024.

1. Per GPU Roofline

- FLOPs per step = ops per param × number of active params = 2 FLOPs × 21 Billion = 42 GFLOPs (forward pass does one multiply and one add)
- Bytes per step = Weight Bytes + KV Cache Bytes = number of active parameters × bytes per param + 1 (MLHA has single KV compressed latent) × number of layers × batch size × sequence length × hidden dimension size × bytes per param
  = 21 Billion ×2 Bytes (GPU must load every element of weight matrix from HBM into SRAM, BF16 is 2 bytes per param) + $2 \times 60 \times 1 \times 1024 \times 576 \times 2$ bytes = 42.14 GB
  (Hidden dimension size = 576 because DeepSeek-V2 uses MHLA instead of MHA)
- Using software efficiency
  - Effective Compute = 989 TFLOPs/s ×0.92 = 910 TFLOPs/s
  - Effective BW = 3.35 TB/s ×0.90 = 3.015 TB/s
- TP = 4 means each GPU holds 1/4 of weights
  - Effective Compute Time = FLOPs per step / (TP × Effective Compute) = $\frac{42 \text{ GFLOPs}}{4 \times 910 \text{ TFLOPs/s}}$ = 0.0115 ms
  - Effective Memory Time = Bytes per step / (TP × Effective BW) = $\frac{42.14 \text{ GB}}{4 \times 3.015 \text{ TB/s}}$ = 3.49 ms
  - memory time > compute time → memory bound
  - Bottleneck Time = max(Effective compute time, Effective memory time) = max(0.0115 ms, 3.49 ms) = 3.49 ms
- Sanity Check via arithmetic intensity
  - Intensity = FLOPs per step / Bytes per step = $\frac{42 \text{ GFLOPs}}{42.14 \text{ GB}}$ = 0.996 FLOPs / byte
  - Ridge Point = $\frac{910 \text{ TFLOPs/s}}{3.015 \text{ TB/s}}$ = 302 FLOPs / byte

- $0.996 \ll 302 \rightarrow$ memory bound

2. AllReduce Communication Overhead (4 GPUs need to sum partial results via ring AllReduce, once after attention, once after FFN)

- Phase 1 - Reduce-Scatter ratio $= (TP-1)/TP = 3/4$
- Phase 2 - All-Gather ratio $= (TP-1)/TP = 3/4$
- Message Bytes = batch size $\times$ sequence length $\times$ hidden dimension size $\times$ bytes per param $= 1 \times 1 \times 5120 \times 2$ bytes = 10 KB (sequence length = 1 for decode)
- AllReduce Time per layer = ((Phase1 + Phase2) $\times$ Message Bytes / NVLink Bandwidth + network injection latency) $\times$ AllReduce Calls per layer (Attention + FFN)

$$= \left(2 \times \frac{TP-1}{TP} \times \frac{10 \text{ KB}}{900 \text{ GB/s}} + 2\ \mu s\right) \times 2 = 4\ \mu s$$

- AllReduce Total Time = AllReduce Time per layer $\times$ # Transformer Layers = 4 micro seconds $\times 60 = 0.24$ ms

3. Parallelism Efficiency

- Efficiency = Bottleneck Time / (Bottleneck Time + AllReduce Total Time) $= \frac{3.49 \text{ ms}}{3.49 \text{ ms} + 0.24 \text{ ms}} = 93.5\%$
- NVLink and NCCL kernel launch is fast enough that the communication overhead is minimal

4. Final ITL

- Inter Token Latency (ITL) = Bottleneck Time / Efficiency $= \frac{3.49 \text{ ms}}{0.935} = 3.73$ ms

5. Conclusion

- TP = 4 gives us almost 4$\times$ lower ITL with minimal communication penalty
- Because we are memory bound by quite a lot we could achieve massive speedup by lowering memory time through better software bandwidth efficiency, hardware with higher HBM, using less transformer layers, reducing output sequence length, shrinking MHLA vector size even further, or using less precise FPs.

Appendix: Equations Generated by Claude Code

Roofline Bound and Efficiency

$$\begin{aligned} \pi_{\text{eff}} &= \pi \cdot \eta_{\text{compute}}, \quad \beta_{\text{eff}} = \beta \cdot \eta_{\text{bw}} \\ P_{\text{achieved}} &= \min(\pi_{\text{eff}},\ \beta_{\text{eff}} \cdot I) \\ \text{MFU} &= \frac{P_{\text{achieved}}}{\pi_{\text{nominal}}} \end{aligned}$$

Workload Arithmetic Intensity (I)

$$
\begin{aligned}
I_{\text{prefill}} &\approx \frac{2BSd^2}{BSd \cdot \text{dtype} + d^2 \cdot \text{dtype}} \\
I_{\text{decode}} &\approx \frac{2}{\text{dtype_bytes}} \\
I_{\text{MoE}} &= \frac{2k \cdot BS \cdot d_{\text{model}} \cdot d_{\text{ffn}}}{k \cdot 2d_{\text{model}} \cdot d_{\text{ffn}} \cdot \text{dtype_bytes}}
\end{aligned}
$$

Parallelism Overhead

$$
\begin{aligned}
t_{\text{AR}} &\approx \frac{2(N-1)}{N} \cdot \frac{M}{\beta_{\text{NVLink}}} + \alpha \\
\eta_{\text{TP}} &= \frac{t_{\text{compute}}}{t_{\text{compute}} + t_{\text{AR}}} \\
L_{\max} &\approx \frac{Bk}{EP} + \sqrt{\frac{Bk}{EP}(1 - k/EP)}\sqrt{2 \ln EP} \\
f_{\text{bubble}} &= \frac{PP - 1}{PP - 1 + m}
\end{aligned}
$$

TCO and Economic Performance

$$
\begin{aligned}
\text{TCO}_{\text{annual}} &= \frac{\text{CapEx}}{D} + (\text{TDP} \cdot N_{\text{GPU}} \cdot \text{PUE} \cdot C_{\text{kWh}} \cdot 8760) + \phi_{\text{opex}}\text{CapEx} \\
C &= \left[\frac{\text{TCO}_{\text{annual}}}{\text{tok/s}_{\text{cluster}} \cdot T_{\text{year}} \cdot \eta_{\text{avail}} \cdot \eta_{\text{util}}}\right] \cdot 10^6
\end{aligned}
$$

FLOP Throughput with TP Example